LINUX / SYSTEMS

Linux Processes Visual Guide

fork/exec, process states, signals, nice values, and zombies.

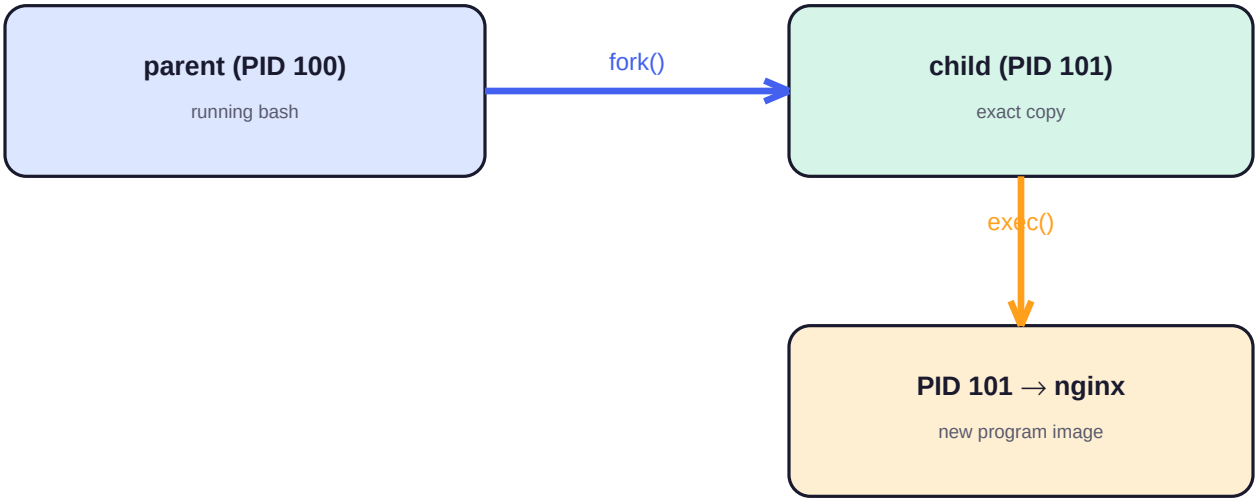
ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

fork() + exec(): How Processes Are Born

Clone yourself, then become someone else.

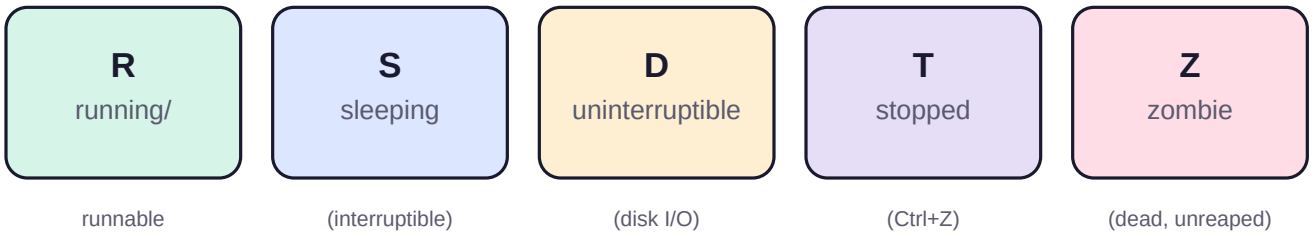


- fork() duplicates the parent: same memory, same file descriptors, new PID.
- exec() replaces the child's memory with a new program — PID stays the same.
- The shell does fork+exec for every command you run. Daemons double-fork to detach.

STEP 2/6

Process States: R S D T Z

What the STAT column in ps really means.



running → sleep (waits) → wake → running … stop (T) → continue → running

exit → ZOMBIE until parent calls wait() → truly gone

D state can't be killed — not even SIGKILL. It's stuck in a kernel syscall (usually storage). Z is already dead: it holds only an exit code, waiting for its parent to collect it.

```
$ ps -eo pid,stat,comm | head
  PID STAT COMMAND
    1 Ss   systemd
  842 Z    [worker] <defunct>
```

STEP 3/6

Signals: Messages to Processes

kill doesn't kill — it SENDS A SIGNAL. Default is TERM.

Signal	#	Default action
SIGTERM	15	polite ask to exit (catchable)
SIGKILL	9	instant death, uncatchable
SIGHUP	1	hangup → daemons reload config
SIGINT	2	Ctrl+C
SIGSTOP	19	pause (uncatchable)
SIGCONT	18	resume
SIGUSR1/2	10/12	app-defined

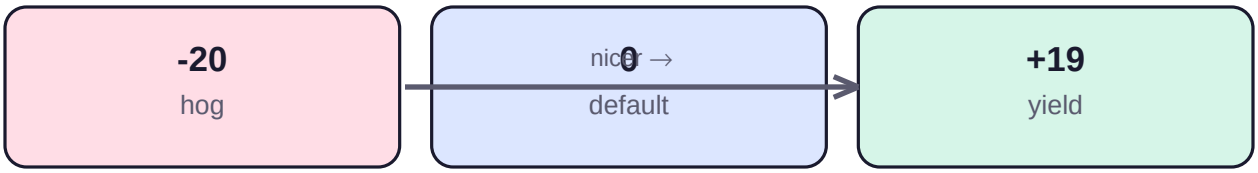
```
$ kill 1234        # SIGTERM — always try this first
$ kill -9 1234     # SIGKILL — last resort, no cleanup
$ kill -HUP 842    # ask daemon to reload config
```

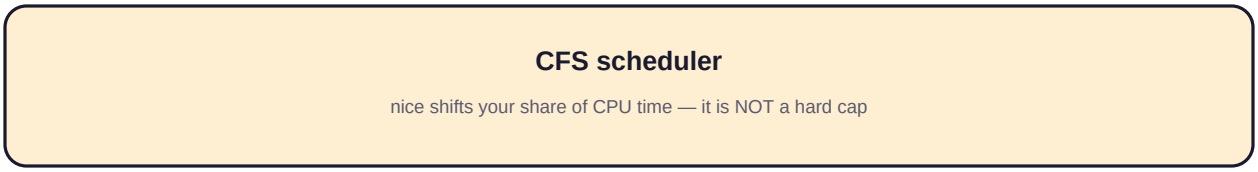
Escalation ladder: TERM → wait → KILL. Jumping straight to -9 skips cleanup handlers and can corrupt state.

STEP 4/6

nice: Who Gets the CPU First

Niceness –20 (greedy) … 0 … +19 (polite). Only root lowers niceness.






```
$ nice -n 10 ./batch-job &     # start polite
$ renice -n 5 -p 1234        # adjust a running process
$ chrt --rr 50 ./rt-task    # real-time class (root): beats ALL nice values
```

STEP 5/6

Zombies & Orphans

Dead-but-listed vs parentless-but-adopted.

- Zombies hold no memory — just a PID and exit code. They vanish when the parent reaps them.
- A pile of zombies means the PARENT is buggy (not calling wait), not the children.
- Cure: fix or restart the parent. Killing zombies is impossible — they're already dead.
- Orphans are harmless: PID 1 adopts them and reaps them properly.

```
$ ps -eo pid,ppid,stat,comm | awk '$3 ~ /Z/'   # find zombies + their parents
```

STEP 6/6

Inspecting Processes

The daily toolkit.

```
$ ps aux --sort=-%cpu | head        # top CPU hogs
$ ps -ef --forest                  # tree view of parents/children
$ top   # or htop: interactive, F9 to signal, F6 to sort
$ pidof nginx  ·  pgrep -af python  # find by name
$ cat /proc/1234/status            # the raw truth
```

- ps aux: one snapshot. top/htop: live view. Learn htop's F-keys — faster than flags.
- /proc/<pid>/ is the source of truth every tool reads: cmdline, environ, fd/, io, status.
- Load average 3 numbers = 1/5/15 min run-queue depth; compare against CPU core count.

STEP ✓/✓

Key Takeaways

Linux Processes

- Every process is born from fork() + exec(); PID 1 (systemd) adopts orphans.
- States: R running, S sleeping, D unkillable I/O wait, T stopped, Z zombie awaiting reaping.
- kill sends signals — escalate TERM → KILL, never start at -9.
- nice –20…+19 shifts CPU share; real-time classes beat nice entirely.
- Zombies are a parent bug, not a child bug — fix the parent, don't chase the dead.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev